

运算与表达式

本章学习目标

- 掌握 Shell 中算术运算的多种实现方式
- 理解数值比较与字符串比较的区别
- 熟练使用逻辑运算符组合条件
- 掌握文件测试运算符的使用
- 学会使用 test 命令和 [] 表达式
- 理解 [] 与 [[]] 的区别及使用场景

4.1 算术运算

4.1.1 Shell 算术运算的特点

Shell 本身是为文本处理设计的，对数学运算的支持相对有限：

特点	说明
默认整数运算	不直接支持小数运算
需要特殊语法	不能直接写 <code>a + b</code> ，需要借助工具
多种实现方式	<code>\$(())</code> 、 <code>let</code> 、 <code>expr</code> 、 <code>bc</code> 等

4.1.2 算术运算符

运算符	说明	示例	结果
+	加法	<code>\$((5 + 3))</code>	8
-	减法	<code>\$((10 - 4))</code>	6
*	乘法	<code>\$((3 * 4))</code>	12
/	除法（取整）	<code>\$((10 / 3))</code>	3
%	取余（模运算）	<code>\$((10 % 3))</code>	1
**	幂运算	<code>\$((2 ** 3))</code>	8

4.1.3 `$(( ))` 双括号（推荐）

这是最常用、最推荐的算术运算方式：

```
#!/bin/bash

a=10
b=3

# 基本运算
echo "加法: $((a + b))"           # 13
echo "减法: $((a - b))"           # 7
echo "乘法: $((a * b))"           # 30
echo "除法: $((a / b))"           # 3（整数除法）
echo "取余: $((a % b))"           # 1
echo "幂运算: $((2 ** 10))"       # 1024

# 复合运算
echo "复合: $(( (a + b) * 2 ))"   # 26

# 直接赋值
result=$((a * b + 5))
echo "结果: $result"             # 35

# 自增自减
((a++))
```

```
echo "a++后: $a"          # 11
((b--))
echo "b--后: $b"          # 2
```

`$(( ))` 的优点:

- 语法简洁，不需要转义
- 可以直接使用变量名（不需要 `$` 前缀）
- 支持所有常见运算符
- 是 Bash 内置功能，效率高

4.1.4 bc 计算器（支持小数）

前面的方法都只支持整数运算，要处理小数需要使用 `bc`：

```
# 基本用法
echo "10 / 3" | bc          # 3（默认整数）

# 指定小数精度
echo "scale=2; 10 / 3" | bc # 3.33

# 更复杂的计算
result=$(echo "scale=4; 3.14159 * 2.5^2" | bc)
echo "圆面积: $result"      # 19.6349

# 使用变量
radius=5
area=$(echo "scale=2; 3.14159 * $radius * $radius" | bc)
echo "半径 $radius 的圆面积: $area" # 78.53
```

bc 常用选项:

设置	说明	示例
<code>scale=n</code>	设置小数位数	<code>scale=2</code> 表示保留2位小数
<code>sqrt(x)</code>	开平方	<code>sqrt(16)</code> = 4
<code>^</code>	幂运算	<code>2^10</code> = 1024

【实操练习 4-1】算术运算综合练习

```
#!/bin/bash
# 文件名: calc_demo.sh
# 功能: 演示各种算术运算方式

echo "===== 算术运算演示 ====="

# 定义变量
num1=100
num2=30

echo "num1 = $num1, num2 = $num2"
echo ""

# 使用 $(( ))
echo "【方式1: \${( ( ) )}】 "
echo "  加法: ${ ( num1 + num2 )}"
echo "  减法: ${ ( num1 - num2 )}"
echo "  乘法: ${ ( num1 * num2 )}"
echo "  除法: ${ ( num1 / num2 )}"
echo "  取余: ${ ( num1 % num2 )}"
echo ""

# 使用 bc (小数运算)
echo "【方式4: bc (小数运算)】 "
echo "  精确除法: $(echo "scale=4; $num1 / $num2" | bc)"
echo "  开平方根: $(echo "scale=2; sqrt($num1)" | bc)"
echo ""

# 实际应用: 计算百分比
total=250
used=175
```

```
percent=$(echo "scale=1; $used * 100 / $total" | bc)
echo "【应用】使用率: ${percent}%"
```

4.2 比较运算

比较运算用于判断两个值之间的关系，是条件判断的基础。

4.2.1 数值比较运算符

在 `[ ]` 或 `test` 命令中使用：

运算符	说明	示例	等价数学符号
<code>-eq</code>	等于 (equal)	<code>[\$a -eq \$b]</code>	<code>==</code>
<code>-ne</code>	不等于 (not equal)	<code>[\$a -ne \$b]</code>	<code>!=</code>
<code>-gt</code>	大于 (greater than)	<code>[\$a -gt \$b]</code>	<code>></code>
<code>-lt</code>	小于 (less than)	<code>[\$a -lt \$b]</code>	<code><</code>
<code>-ge</code>	大于等于 (greater or equal)	<code>[\$a -ge \$b]</code>	<code>>=</code>
<code>-le</code>	小于等于 (less or equal)	<code>[\$a -le \$b]</code>	<code><=</code>

记忆技巧：

```
-eq  →  equal      等于
-ne  →  not equal   不等于
-gt  →  greater than 大于
-lt  →  less than   小于
-ge  →  greater or equal 大于等于
-le  →  less or equal 小于等于
```

【实操练习 4-2】数值比较演示

```
#!/bin/bash
```

```
# 文件名: num_compare.sh

a=10
b=20

echo "a = $a, b = $b"
echo ""

# 各种比较
if [ $a -eq $b ]; then
    echo "a 等于 b"
else
    echo "a 不等于 b"
fi

if [ $a -lt $b ]; then
    echo "a 小于 b"
fi

if [ $a -le 10 ]; then
    echo "a 小于等于 10"
fi

# 结合实际应用
disk_usage=85
if [ $disk_usage -gt 80 ]; then
    echo "警告: 磁盘使用率 ${disk_usage}% 超过 80%! "
fi
```

4.2.2 在 (()) 中使用数学符号

在双括号 (()) 中可以直接使用数学比较符号：

```
a=10
b=20

# (( )) 返回值可直接用于条件判断
(( a > b )) && echo "a > b" || echo "a <= b"
```

4.2.3 字符串比较运算符

字符串比较使用不同的运算符：

运算符	说明	示例
<code>=</code> 或 <code>==</code>	字符串相等	<code>["\$str1" = "\$str2"]</code>
<code>!=</code>	字符串不相等	<code>["\$str1" != "\$str2"]</code>
<code>-z</code>	字符串为空（长度为0）	<code>[-z "\$str"]</code>
<code>-n</code>	字符串非空（长度不为0）	<code>[-n "\$str"]</code>
<code><</code>	字典序小于（需转义或用[[]]	<code>[["\$str1" < "\$str2"]]</code>
<code>></code>	字典序大于（需转义或用[[]]	<code>[["\$str1" > "\$str2"]]</code>

 **重要：**字符串比较时，变量一定要用双引号括起来！

为什么要用双引号？

```
str=""

# 不加引号（危险！）
[ $str = "hello" ]      # 展开后变成 [ = "hello" ]，语法错误！

# 加引号（安全）
[ "$str" = "hello" ]    # 展开后变成 [ "" = "hello" ]，正确
```

【实操练习 4-3】字符串比较演示

```
#!/bin/bash
# 文件名: str_compare.sh

str1="hello"
str2="world"
str3=""
```

```

str4=" "

echo "str1 = '$str1'"
echo "str2 = '$str2'"
echo "str3 = '$str3' (空字符串) "
echo ""

# 相等比较
if [ "$str1" = "$str2" ]; then
    echo "str1 等于 str2"
else
    echo "str1 不等于 str2"
fi

# 空字符串检查
if [ -z "$str3" ]; then
    echo "str3 是空字符串"
fi

if [ -n "$str1" ]; then
    echo "str1 非空"
fi

# 实际应用：检查用户输入
read -p "请输入用户名: " username
if [ -z "$username" ]; then
    echo "错误：用户名不能为空！"
    exit 1
fi
echo "你好, $username! "

```

4.2.4 数值比较 vs 字符串比较

这是初学者容易混淆的地方：

```

# 数值比较（比较数学大小）
[ 5 -lt 10 ]      # true, 5 小于 10

# 字符串比较（比较字典序）
[[ "5" < "10" ]] # false! 字符 '5' > '1' (字典序)

```



```
# 正确示例
a=5
b=10

# 作为数字比较
if [ $a -lt $b ]; then
    echo "$a 小于 $b (数值比较) "
fi

# 作为字符串比较
if [[ "$a" > "$b" ]]; then
    echo "'$a' 大于 '$b' (字典序比较) "
fi
```

4.3 逻辑运算

逻辑运算用于组合多个条件。

4.3.1 逻辑运算符

在 `[]` 或 `(( ))` 中使用：

运算符	说明	示例
<code>&&</code>	逻辑与 (AND)	<code>[[\$a -gt 0 && \$a -lt 10]]</code>
<code> </code>	逻辑或 (OR)	<code>[[\$a -lt 0 \$a -gt 10]]</code>
<code>!</code>	逻辑非 (NOT)	<code>[[! -f file.txt]]</code>

连接多个 `[]` 条件：

```
# 使用 && 和 || 连接多个 [ ] 表达式
[ $a -gt 0 ] && [ $a -lt 10 ]      # a > 0 且 a < 10
[ $a -lt 0 ] || [ $a -gt 10 ]     # a < 0 或 a > 10
```

【实操练习 4-4】逻辑运算演示

```
#!/bin/bash
# 文件名: logic_demo.sh

age=25
score=85

echo "年龄: $age, 分数: $score"
echo ""

# 使用 && (与) - 推荐方式
if [[ $age -ge 18 && $score -ge 60 ]]; then
    echo "【&&】成年且及格"
fi

# 使用 ! (非)
file="/tmp/test.txt"
if [ ! -f "$file" ]; then
    echo "【!】文件 $file 不存在"
fi

# 复杂条件组合
if [[ $age -ge 18 && $age -le 60 && $score -ge 60 ]]; then
    echo "【复合条件】18-60岁之间且及格"
fi
```

4.3.2 短路求值

逻辑运算符 `&&` 和 `||` 支持短路求值，常用于简化条件执行：

```
# && 短路：前面为真才执行后面
[ -f /etc/passwd ] && echo "文件存在"

# || 短路：前面为假才执行后面
[ -f /nonexistent ] || echo "文件不存在"

# 组合使用（三元运算符效果）
[ $age -ge 18 ] && echo "成年" || echo "未成年"

# 实际应用
```

```
mkdir -p /tmp/mydir && cd /tmp/mydir && echo "成功进入目录"

# 检查命令是否存在
command -v git > /dev/null && echo "Git 已安装" || echo "Git 未安装"
(谨慎使用)
```

4.4 文件测试运算符

文件测试运算符用于检查文件的各种属性，这在脚本中非常常用。

4.4.1 文件存在与类型检测

运算符	说明	示例
-e	文件存在（任何类型）	[-e /path/file]
-f	是普通文件	[-f /path/file]
-d	是目录	[-d /path/dir]
-L	是符号链接	[-L /path/link]
-b	是块设备	[-b /dev/sda]
-c	是字符设备	[-c /dev/tty]
-S	是套接字	[-S /var/run/socket]
-p	是命名管道	[-p /path/pipe]

4.4.2 文件权限检测

运算符	说明	示例
<code>-r</code>	可读	<code>[-r file]</code>
<code>-w</code>	可写	<code>[-w file]</code>
<code>-x</code>	可执行	<code>[-x file]</code>
<code>-u</code>	设置了 SUID	<code>[-u file]</code>
<code>-g</code>	设置了 SGID	<code>[-g file]</code>
<code>-k</code>	设置了粘滞位	<code>[-k dir]</code>

4.4.3 文件属性检测

运算符	说明	示例
<code>-s</code>	文件大小大于0（非空）	<code>[-s file]</code>
<code>-N</code>	文件自上次读取后被修改	<code>[-N file]</code>
<code>-O</code>	文件属于当前用户	<code>[-O file]</code>
<code>-G</code>	文件属于当前用户组	<code>[-G file]</code>

4.4.4 文件比较

运算符	说明	示例
<code>-nt</code>	更新（newer than）	<code>[file1 -nt file2]</code>
<code>-ot</code>	更旧（older than）	<code>[file1 -ot file2]</code>
<code>-ef</code>	同一文件（硬链接）	<code>[file1 -ef file2]</code>

【实操练习 4-5】文件测试运算符演示

```
#!/bin/bash
# 文件名: file_test.sh

# 创建测试环境
test_dir="/tmp/file_test_demo"
mkdir -p "$test_dir"
touch "$test_dir/normal_file.txt"
echo "hello" > "$test_dir/non_empty.txt"
mkdir "$test_dir/subdir"
ln -sf "$test_dir/normal_file.txt" "$test_dir/link_file"
chmod 755 "$test_dir/non_empty.txt"

echo "==== 文件测试演示 ====="
echo ""

# 存在性和类型测试
file="$test_dir/normal_file.txt"
echo "测试文件: $file"

[ -e "$file" ] && echo "    ✓ 文件存在 (-e)"
if [ -e "$file" ]; then
    echo "    ✓ 文件存在 (-e)"
fi
[ -f "$file" ] && echo "    ✓ 是普通文件 (-f)"
[ -d "$test_dir/subdir" ] && echo "    ✓ subdir 是目录 (-d)"
[ -L "$test_dir/link_file" ] && echo "    ✓ link_file 是符号链接 (-L)"

echo ""

# 权限测试
file2="$test_dir/non_empty.txt"
echo "测试文件: $file2"

[ -r "$file2" ] && echo "    ✓ 可读 (-r)"
[ -w "$file2" ] && echo "    ✓ 可写 (-w)"
[ -x "$file2" ] && echo "    ✓ 可执行 (-x)"

echo ""

# 内容测试
[ -s "$file" ] || echo "    o normal_file.txt 是空文件"
```

```
[ -s "$file2" ] && echo "    ✓ non_empty.txt 有内容 (-s)"

echo ""

# 实际应用示例
config_file="/etc/bashrc"
if [ -f "$config_file" ] && [ -r "$config_file" ]; then
    echo "配置文件 $config_file 存在且可读"
else
    echo "无法读取配置文件"
fi

# 清理
rm -rf "$test_dir"
echo ""
echo "测试完成，已清理临时文件"
```

4.5 test 命令与 [] 表达式

4.5.1 test 命令

`test` 是一个内置命令，用于条件测试：

```
# test 命令语法
test expression

# 示例
test -f /etc/passwd = [ -f /etc/passwd ]
echo $?      # 0 表示真（文件存在）

test 5 -gt 3
echo $?      # 0 表示真

test "abc" = "def"
echo $?      # 1 表示假
```

4.5.2 [] 是 test 的简写

`[ ]` 实际上是 `test` 命令的另一种写法，更加直观：

```
# 以下两种写法完全等价
test -f /etc/passwd
[ -f /etc/passwd ]

# 注意：[ 后面和 ] 前面必须有空格！
[ -f /etc/passwd ]      # 正确
[-f /etc/passwd]       # 错误！
```

 **重要：** `[` 实际上是一个命令，`]` 是它的参数，所以必须用空格隔开。

```
# 查看 [ 命令
type [
# 输出：[ is a shell builtin

which [
# 输出：/usr/bin/[
```

4.5.3 `[ ]` 使用注意事项

```
# 1. 方括号内侧必须有空格
[ $a -eq $b ]      # 正确
[$a -eq $b]       # 错误

# 2. 变量要用双引号
[ "$str" = "hello" ]      # 正确（推荐）
[ $str = "hello" ]       # 危险（变量为空时会出错）

# 3. 运算符两边要有空格
[ $a -eq $b ]      # 正确
[ $a-eq$b ]       # 错误

# 4. 字符串比较用 = 而不是 ==（虽然 == 在 bash 中也可以）
[ "$str" = "hello" ]      # POSIX 标准
[ "$str" == "hello" ]     # Bash 扩展
```

4.6 [[]] 扩展测试

`[[ ]]` 是 Bash 的扩展语法，比 `[ ]` 更强大和安全。

4.6.1 [[]] 的优势

特性	[]	[[]]
单词拆分	会发生	不会发生
通配符展开	会发生	不会发生
正则匹配	不支持	支持 <code>=~</code>
模式匹配	不支持	支持 <code>==</code> 配合通配符
逻辑运算符	<code>-a</code> , <code>-o</code>	<code>&&</code> , <code> </code>
<code><</code> <code>></code> 比较	需要转义	直接使用

4.6.2 [[]] 示例

```
#!/bin/bash

str="hello world"
filename="report_2024.txt"

# 1. 不需要给变量加引号（但建议加）
[[ $str = "hello world" ]] && echo "匹配成功"

# 2. 支持模式匹配（通配符）
[[ $filename == *.txt ]] && echo "是txt文件"
[[ $filename == report_* ]] && echo "是report文件"

# 3. 支持正则表达式匹配
email="user@example.com"
if [[ $email =~ ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$ ]];
then
```



```
    echo "有效的邮箱格式"
fi

# 4. 直接使用 && 和 ||
age=25
if [[ $age -ge 18 && $age -le 60 ]]; then
    echo "工作年龄"
fi

# 5. 字符串比较不需要转义 < >
[[ "apple" < "banana" ]] && echo "apple 在 banana 前面（字典序）"
```

4.6.4 [] 与 [[]] 对比总结

```
# 推荐使用 [[ ]] 的场景：
# 1. 字符串比较
[[ "$str" = "hello" ]]

# 2. 模式匹配
[[ "$file" == *.sh ]]

# 3. 正则匹配
[[ "$email" =~ @.*\.com$ ]]

# 4. 复杂条件
[[ $a -gt 0 && $a -lt 100 ]]

# 使用 [ ] 的场景：
# 1. 需要 POSIX 兼容性（如 sh 脚本）
# 2. 简单的文件测试
[ -f /etc/passwd ]
```

4.7 综合实战：系统健康检查脚本

让我们综合运用本章学到的知识，编写一个系统健康检查脚本。

【实操练习 4-6】系统健康检查脚本

```
#!/bin/bash
#####
# 文件名: health_check.sh
# 功能描述: 系统健康检查脚本
# 使用方法: ./health_check.sh
#####

# 颜色定义
RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[0;33m'
NC='\033[0m' # No Color

# 阈值设置
readonly CPU_THRESHOLD=0
readonly MEM_THRESHOLD=0
readonly DISK_THRESHOLD=0

# 状态计数
warnings=0
errors=0

#-----
# 打印函数
#-----
print_header() {
    echo ""
    echo "===== "
    echo "        系统健康检查报告"
    echo "        检查时间: $(date '+%Y-%m-%d %H:%M:%S')"
    echo "===== "
    echo ""
}

print_ok() {
    echo -e "${GREEN}[OK]${NC} $1"
}

print_warn() {
    echo -e "${YELLOW}[警告]${NC} $1"
    ((warnings++))
}
```

```

print_error() {
    echo -e "${RED}[错误]${NC} $1"
    ((errors++))
}

#-----
# 检查 CPU 使用率
#-----

check_cpu() {
    echo "【CPU 检查】"

    # 获取 CPU 使用率 (取 1 分钟负载)
    local cpu_cores=$(grep -c processor /proc/cpuinfo)
    local load_avg=$(cat /proc/loadavg | awk '{print $1}')
    local cpu_usage=$(echo "scale=0; $load_avg * 100 / $cpu_cores" |
bc)

    echo "   CPU 核心数: $cpu_cores"
    echo "   1分钟负载: $load_avg"
    echo "   负载百分比: ${cpu_usage}%"

    if (( cpu_usage >= CPU_THRESHOLD )); then
        print_warn "CPU 负载过高 (>= ${CPU_THRESHOLD}%) "
    else
        print_ok "CPU 负载正常"
    fi
    echo ""
}

#-----
# 检查内存使用率
#-----

check_memory() {
    echo "【内存检查】"

    # 获取内存信息
    local mem_total=$(free -m | awk '/^Mem:/ {print $2}')
    local mem_used=$(free -m | awk '/^Mem:/ {print $3}')
    local mem_usage=$((mem_used * 100 / mem_total))

    echo "   总内存: ${mem_total}MB"

```

```

echo "  已使用: ${mem_used}MB"
echo "  使用率: ${mem_usage}%"

if [ $mem_usage -ge $MEM_THRESHOLD ]; then
    print_warn "内存使用率过高 (>= ${MEM_THRESHOLD}%)"
else
    print_ok "内存使用率正常"
fi
echo ""
}

#-----
# 检查磁盘使用率
#-----
check_disk() {
    echo "【磁盘检查】"

    local has_warning=0

    # 检查每个挂载点
    while read -r line; do
        local device=$(echo "$line" | awk '{print $1}')
        local mount=$(echo "$line" | awk '{print $6}')
        local usage=$(echo "$line" | awk '{print $5}' | tr -d '%')

        echo "  $mount ($device): ${usage}%"

        if [ $usage -ge $DISK_THRESHOLD ]; then
            print_warn "$mount 磁盘使用率过高 (>= ${DISK_THRESHOLD}%)"
            has_warning=1
        fi
    done < <(df -h | grep '^/dev/')

    if [ $has_warning -eq 0 ]; then
        print_ok "所有磁盘使用率正常"
    fi
    echo ""
}

#-----
# 检查关键服务
#-----

```

```

check_services() {
    echo "【服务检查】"

    # 定义要检查的服务列表
    local services=("sshd" "crond")

    for svc in "${services[@]}; do
        if systemctl is-active "$svc" > /dev/null 2>&1; then
            print_ok "$svc 服务运行中"
        else
            print_error "$svc 服务未运行"
        fi
    done
    echo ""
}

#-----
# 检查关键文件
#-----
check_files() {
    echo "【关键文件检查】"

    local files=("/etc/passwd" "/etc/shadow" "/etc/hosts")

    for f in "${files[@]}; do
        if [ -f "$f" ] && [ -r "$f" ]; then
            print_ok "$f 存在且可读"
        else
            print_error "$f 不存在或不可读"
        fi
    done
    echo ""
}

#-----
# 检查网络连接
#-----
check_network() {
    echo "【网络检查】"

    # 检查本地回环
    if ping -c 1 127.0.0.1 > /dev/null 2>&1; then

```

```

        print_ok "本地回环正常"
    else
        print_error "本地回环异常"
    fi

    # 检查网关连接
    local gateway=$(ip route | grep default | awk '{print $3}' | head
-1)
    if [ -n "$gateway" ]; then
        if ping -c 1 -W 2 "$gateway" > /dev/null 2>&1; then
            print_ok "网关 $gateway 可达"
        else
            print_warn "网关 $gateway 不可达"
        fi
    else
        print_warn "未找到默认网关"
    fi
    echo ""
}

#-----
# 生成检查报告
#-----
print_summary() {
    echo "=====
    echo "                检查总结"
    echo "=====
    echo ""

    if [ $errors -gt 0 ]; then
        echo -e "${RED}发现 $errors 个错误${NC}"
    fi

    if [ $warnings -gt 0 ]; then
        echo -e "${YELLOW}发现 $warnings 个警告${NC}"
    fi

    if [ $errors -eq 0 ] && [ $warnings -eq 0 ]; then
        echo -e "${GREEN}所有检查项通过，系统状态良好! ${NC}"
    fi

    echo ""
}

```

```

    echo "===== "
}

#-----
# 主程序
#-----
main() {
    print_header
    check_cpu
    check_memory
    check_disk
    check_services
    check_files
    check_network
    print_summary

    # 返回适当的退出码
    if [ $errors -gt 0 ]; then
        exit 2
    elif [ $warnings -gt 0 ]; then
        exit 1
    else
        exit 0
    fi
}

# 执行主程序
main

```

执行脚本：

```

chmod +x ~/scripts/health_check.sh
~/scripts/health_check.sh

```

预期输出效果：

```

=====
      系统健康检查报告
      检查时间：2024-01-15 10:30:00
=====

```

【CPU 检查】

CPU 核心数：2

1分钟负载：0.15

负载百分比：7%

[OK] CPU 负载正常

【内存检查】

总内存：1837MB

已使用：512MB

使用率：27%

[OK] 内存使用率正常

【磁盘检查】

/ (/dev/sda1): 45%

[OK] 所有磁盘使用率正常

【服务检查】

[OK] sshd 服务运行中

[OK] crond 服务运行中

【关键文件检查】

[OK] /etc/passwd 存在且可读

[OK] /etc/shadow 存在且可读

[OK] /etc/hosts 存在且可读

【网络检查】

[OK] 本地回环正常

[OK] 网关 192.168.1.1 可达

检查总结

所有检查项通过，系统状态良好！

4.8 本章小结

本章我们学习了以下核心内容：

算术运算：

1. `$(( ))` 是最推荐的整数运算方式
2. `bc` 支持小数运算

比较运算：

1. 数值比较使用 `-eq`、`-ne`、`-gt`、`-lt`、`-ge`、`-le`
2. 字符串比较使用 `=`、`!=`、`-z`、`-n`
3. 数值比较和字符串比较不能混用

逻辑运算：

1. `[ ]` 中使用 `-a`、`-o`
2. `[ [ ] ]` 或条件连接使用 `&&`、`||`
3. `!` 用于逻辑非

文件测试：

1. `-e`、`-f`、`-d` 检测存在和类型
2. `-r`、`-w`、`-x` 检测权限
3. `-s` 检测文件是否非空

`[ ]` 与 `[ [ ] ]`：

1. `[ ]` 是 POSIX 标准，兼容性好
 2. `[ [ ] ]` 是 Bash 扩展，功能更强
 3. `[ [ ] ]` 支持模式匹配和正则匹配
-

4.9 课后练习

练习题 1（基础）：编写脚本，计算并输出 $1+2+3+\dots+100$ 的和。

练习题 2（基础）：编写脚本，接收两个数字参数，输出它们的和、差、积、商（保留2位小数）。

练习题 3（进阶）：编写脚本，接收一个文件路径作为参数：

- 检查文件是否存在
- 如果存在，判断是文件还是目录
- 检查是否可读、可写、可执行
- 如果是文件，输出文件大小是否为空

练习题 4（进阶）：编写脚本，验证用户输入的内容：

- 检查是否为空
- 检查是否为纯数字
- 检查数字是否在 1-100 范围内

练习题 5（综合）：扩展系统健康检查脚本，添加以下功能：

- 检查系统运行时间，如果超过 30 天给出提示
- 检查 /tmp 目录大小，超过 1GB 给出警告
- 将检查结果保存到日志文件